

Indian Institute of Technology Delhi

Department of Computer Science and Engineering

COD7001: Cornerstone Project

Lab 4: Bytecode Virtual Machine

Chirag Kathpalia (2025MCS2098)

Aditya Arya (2025MCS2114)

Supervisor: Prof. Kolin Paul

1 Architecture of the VM

1.1 Overall Execution Model

The VM follows a **stack-based execution model**, where all computations operate on an explicit operand stack rather than registers. Instructions implicitly consume operands from the top of the stack and push results back, simplifying instruction design and execution.

Program execution follows a standard **fetch–decode–execute loop** driven by the Program Counter (PC). The PC points to the next instruction in the bytecode array and is normally advanced sequentially, while control-flow instructions explicitly modify it to implement jumps, conditionals, and function calls. The VM ensures safe execution by validating that the PC always remains within the bounds of the loaded bytecode.

1.2 Stack Design

The VM uses a **unified stack architecture**, where a single stack stores both operand data and control-flow metadata. During execution, the stack holds operands, intermediate results, function arguments, call-frame metadata (return address, saved frame pointer, argument count), and return values.

Stack access is managed using a **stack pointer (sp)**, which tracks the next free slot, and a **frame pointer (fp)**, which identifies the base of the current function’s activation record. The frame pointer always refers to the return-address slot of the current frame, enabling reliable access to function arguments even as the stack grows and shrinks.

Function calls push the return address, previous frame pointer, and argument count onto the stack before updating **fp** and transferring control. Returns restore the caller’s execution state by unwinding the stack frame and pushing the return value back onto the stack. This design supports nested and recursive calls while keeping the runtime implementation simple.

A limitation of this approach is that operand data and control metadata share the same stack, requiring careful stack layout discipline by the assembler and bytecode programs.

1.3 Memory Layout

The VM memory model consists of three regions:

- **Code memory**, which stores the bytecode program and is indexed by the Program Counter
- **Stack memory**, a fixed-size array used for operands, call frames, and return values
- **Global data segment**, a fixed-size array of 32-bit integers accessed using indexed load and store operations

All memory accesses are bounds-checked. Fixed limits on the stack and global segment simplify memory management, avoid dynamic allocation during execution, and make runtime behavior predictable.

1.4 Safety & Bounds

Runtime safety is enforced through explicit bounds checks on all critical operations. Stack overflow and underflow are detected during push and pop operations and cause immediate termination with a clear error message. Similarly, the VM validates the Program Counter before every instruction fetch to prevent execution outside the code segment.

These checks ensure that invalid memory access or corrupted control flow cannot occur silently. As a result, the VM either executes correctly or fails fast with well-defined error behavior, preserving correctness and debuggability.

2 Types of Instructions and Instruction Dispatch Strategy

- Instructions are **fetches sequentially** from the bytecode array using the Program Counter (PC), which always points to the next opcode to execute.
- Instruction **decoding and dispatch** is implemented using a **switch** statement over the opcode value, mapping each opcode directly to its execution logic.
- Operands are handled in two ways:
 - **Immediate operands** are embedded in the bytecode and read by advancing the PC.
 - **Stack operands** are implicitly consumed from and produced onto the operand stack.
- This dispatch strategy was chosen for its **simplicity, clarity, and ease of debugging**, making it well-suited for an educational VM.
- The design allows **explicit control-flow handling**, straightforward bounds checking, and clear reasoning about runtime behavior.
- The main drawback is **lower performance** due to repeated branch checks in the **switch** dispatch loop.
- More advanced alternatives such as **jump tables** or **threaded dispatch** could reduce dispatch overhead, but were not used to preserve **portability, readability, and maintainability**.

2.1 Instruction Set Architecture (ISA)

The VM implements a fixed, byte-oriented instruction set. Each instruction opcode is exactly **1 byte**, optionally followed by operands encoded directly in the bytecode. This instruction set is a *contract* shared by the VM, assembler, and test programs.

2.1.1 Instruction Table

- The instruction set is **stack-oriented**, with most instructions implicitly operating on stack operands.
- Control-flow instructions explicitly modify the Program Counter.
- Local-variable access is supported through indexed addressing relative to the current frame pointer.
- Opcodes are fixed and architecture-independent, similar in spirit to real ISAs where opcode values form a stable contract across implementations.

Instruction	Opcode (Hex)	Operands	Description
NOP	0x00	—	No operation; advances PC
PUSH	0x01	<int32>	Push immediate 32-bit value onto stack
DUP	0x03	—	Duplicate top of stack
SWAP	0x04	—	Swap top two stack values
DROP	0x05	—	Remove top of stack
OVER	0x06	—	Push second stack value to top
ADD	0x10	—	Pop two values, push $a + b$
SUB	0x11	—	Pop two values, push $a - b$
MUL	0x12	—	Pop two values, push $a * b$
DIV	0x13	—	Pop two values, push a / b
INC	0x16	—	Increment top of stack
DEC	0x17	—	Decrement top of stack
NEG	0x18	—	Negate top of stack
EQ	0x19	—	Push 1 if equal, else 0
LT	0x1A	—	Push 1 if less-than, else 0
GT	0x1B	—	Push 1 if greater-than, else 0
JMP	0x20	<addr>	Unconditional jump to address
JZ	0x21	<addr>	Jump if popped value is zero
STORE	0x30	<idx>	Store value into global index
LOAD	0x31	<idx>	Load value from global index
STORE_LOCAL	0x32	<idx>	Store value into local argument slot
LOAD_LOCAL	0x33	<idx>	Load value from local argument slot
CALL	0x40	<addr> <nargs>	Call function with nargs arguments
RET	0x41	—	Return from function
PRINT	0x60	—	Pop and print value
HALT	0xFF	—	Stop VM execution

Table 1: Complete Instruction Set

3 Design of Call Frames and Return Mechanism

The VM implements function calls using a **unified stack** with explicit frame management. Control-flow metadata and function arguments are stored directly on the operand stack, and correctness depends on a disciplined calling convention followed by the assembler and bytecode programs.

3.1 Call Semantics

When a CALL instruction executes:

- The target function address and argument count (**nargs**) are read from the bytecode.
- The VM pushes the following metadata onto the stack, in order:
 - Return address (current PC)
 - Saved frame pointer (**fp**)
 - Number of arguments (**nargs**)
- The frame pointer is updated to point to the **return address slot** of the new frame.
- Control jumps to the callee’s entry address.

Function arguments are **not pushed by CALL itself**. They are expected to already be present on the stack below the frame, placed there by the caller prior to invocation.

3.2 Frame Layout and Local Access

Each stack frame consists of:

- Return address
- Saved frame pointer
- Argument count (**nargs**)
- Function arguments (stored below the frame)

The frame pointer (**fp**) always points to the **return address slot**, providing a stable reference point even as the stack pointer (**sp**) changes during execution.

Argument access is performed relative to **fp**:

- **LOAD_LOCAL i** first reads **nargs** from **stack[fp + 2]**
- The argument slot is computed as:

$$\text{slot} = \text{fp} - \text{nargs} + i \quad (1)$$

This calculation allows arguments to be accessed reliably despite intermediate stack pushes and pops.

3.3 Return Semantics

When a **RET** instruction executes:

- The function's return value is popped from the stack.
- The VM reads:
 - **nargs**
 - Saved frame pointer
 - Return address
- The stack pointer is rewound to remove:
 - Frame metadata
 - Function arguments
- The previous frame pointer and program counter are restored.
- The return value is pushed back onto the stack for the caller.

This mechanism supports nested and recursive calls while preserving correct control flow.

3.4 Issues Encountered During Development

Several real issues emerged while implementing this design:

- Frame corruption due to incorrect stack cleanup
- Off-by-one errors in argument indexing
- Incorrect access to arguments during deep recursion
- Infinite recursion caused by misaligned frame pointers

Recursive programs such as **factorial** and deeply nested calls were particularly effective in exposing these bugs. These issues highlight a key constraint of the unified stack design: operand data and control metadata coexist, making correctness highly sensitive to stack discipline and assembler correctness.

4 Limitations and Possible Enhancements

4.1 Limitations

- The VM uses a **unified stack**, where operand data, function arguments, and call-frame metadata coexist.
- Correct execution depends on **strict calling conventions** enforced by the assembler and bytecode.
- **Deep recursion and nested function calls** cause rapid stack growth, making stack overflow the primary failure mode under stress.
- Recursive workloads are relatively expensive due to repeated **frame setup and tear-down**.
- The absence of **tail-call optimization** further increases recursion overhead.
- Interleaving operand operations with control metadata makes the stack **harder to reason about**, increasing the risk of frame corruption.
- The VM has **no heap or dynamic memory allocation**, limiting programs to stack- and global-only data.
- Fixed stack and global sizes restrict scalability for large or long-running programs.

4.2 Possible Enhancements

- Separate the **operand stack and call stack** to simplify frame management and eliminate metadata interference.
- Introduce a **heap with basic allocation primitives** to support dynamic data structures.
- Implement **tail-call optimization** to improve performance for recursive programs.
- Improve instruction dispatch using **jump tables or threaded dispatch** to reduce decode overhead.
- Add richer **local variable support** and clearer frame layouts.
- Include **debugging metadata** (stack traces, frame inspection) for easier diagnosis.
- Explore a **simple garbage collector** to make the VM closer to real-world runtimes while retaining educational clarity.